Studio Nx

Backend(FastAPI/Python) 51 — check_rate_limit() · record_inquiry_send()

2026-10-12

Backend(FastAPI/Python) 51 — `check_rate_limit()` · `record_inquiry_send()`

항목	내용
파일	`studionx/backend/app/services/inquiry_service.py`
작업	P1-12 문의 API: 반복 전송 제한 (PRD 01 FR-113)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	50 `InquiryCreateRequest` · 52 `create_inquiry()`

1. 하는 일

같은 사람이 짧은 시간 안에 문의를 계속 보내지 못하게 막습니다. **같은 이메일이나 같은 IP로 10분 안에 2번** 저장되었다면, 3번째는 429로 응답합니다.

상황 (같은 이메일, 같은 IP)	결과
1번째, 2번째	통과합니다
3번째 (10분 안)	429 `TOO_MANY_REQUESTS` "Too many messages. Please try again later."이고, `details.retry_after`에 몇 초 뒤에 보낼 수 있는지 담깁니다
IP는 다르지만 이메일이 같음 (`A@Example.com` 도 같은 이메일로 봅니다)	막힙니다
이메일은 다르지만 IP가 같음	막힙니다
IP도 이메일도 다름	통과합니다
저장에 **실패한** 시도와 막힌 시도	**세지 않습니다**
첫 번째 저장으로부터 10분이 지남	한 자리가 비어서 다시 1번 보낼 수 있습니다

2. 언제 쓰나

52번 `create_inquiry` 안에서 다음 순서로 부릅니다.

순서	함수
① 저장하기 전	`check_rate_limit(ip, email)`: 이미 2번 저장되었다면 여기서 429로 응답합니다 (저장하지 않습니다)
② 저장(commit)에 성공한 뒤	`record_inquiry_send(ip, email)`: 이번 전송을 셉니다

사용자 결정 (2026-10-08)

정한 것	결정	템플릿
횟수와 시간	**10분에 2번**	10분에 3번
저장에 실패한 시도도 세는가	**세지 않음**	(정하기)

템플릿의 `check_rate_limit`는 함수 하나가 "검사하고, 통과하면 바로 센다"는 구조였습니다. 그렇게 하면 검사를 통과한 뒤 **저장에 실패해도** 이미 세어진 상태가 됩니다. 결정대로 하려면 세는 일을 저장 **뒤로** 옮겨야 해서, 함수를 둘로 나눴습니다.

기록은 메모리에 두는 방식이며, 이것은 2026-10-03 결정을 그대로 따른 것입니다. 그래서 서버를 다시 켜면 기록이 비고, 서버가 여러 대라면 각자 따로 셉니다 (Phase 1은 서버 한 대).

3. 코드

import (파일 맨 위)

```
import threading
import time
from collections import deque

from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Inquiry, User
from app.schemas.inquiry import InquiryCreateRequest
```

`threading`, `time`, `AppError`, `ErrorCode`를 이번에 추가했습니다.

상수와 기록 장소

```
INQUIRY_LIMIT_COUNT = 2  # 이 횟수까지 (2026-10-08 사용자 결정: 10분에 2번 — 템플릿은 3)
INQUIRY_LIMIT_SECONDS = 10 * 60  # 이 시간(초) 안에

_recent_sends: dict[str, deque[float]] = {}
_lock = threading.Lock()  # FastAPI 동기 함수는 여러 스레드에서 돎 → 기록을 함께 고치지 않게
_clock = time.monotonic  # 시스템 시계를 바꿔도 영향 없는 시각 (시험에서 바꿔 끼움)


def _keys(ip: str | None, email: str) -> list[str]:
    """제한을 셀 열쇠 — 이메일(소문자) 하나 + IP 가 있으면 IP 하나."""
    keys = [f"email:{email.strip().lower()}"]
    if ip:
        keys.append(f"ip:{ip}")
    return keys
```

함수

```
def check_rate_limit(ip: str | None, email: str) -> None:
    now = _clock()
    with _lock:
        for key in _keys(ip, email):
            q = _recent_sends.get(key)
            if q is None:
                continue
            while q and now - q[0] > INQUIRY_LIMIT_SECONDS:
                q.popleft()
            if not q:
                del _recent_sends[key]
                continue
            if len(q) >= INQUIRY_LIMIT_COUNT:
                raise AppError(
                    ErrorCode.TOO_MANY_REQUESTS,
                    429,
```

```
                    "Too many messages. Please try again later.",
                    details={"retry_after": int(INQUIRY_LIMIT_SECONDS - (now - q[0])) + 1},
                )


def record_inquiry_send(ip: str | None, email: str) -> None:
    now = _clock()
    with _lock:
        for key in _keys(ip, email):
            _recent_sends.setdefault(key, deque()).append(now)
```

문법 풀이

import 문법	뜻
`import threading`	여러 스레드가 같은 자료를 동시에 고치지 못하게 하는 잠금(`Lock`)을 씁니다
`import time`	`time.monotonic()`은 컴퓨터가 켜진 뒤 흐른 초입니다. 시계를 바꿔도 뒤로 가지 않습니다
`from collections import deque`	양쪽 끝에서 넣고 빼기가 빠른 목록입니다. 오래된 기록은 앞에서 빼고(`popleft`), 새 기록은 뒤에 넣습니다(`append`)

식	뜻
`dict[str, deque[float]]`	열쇠(`"email:a@b.com"`)마다 저장 시각 목록을 둡니다
`with _lock:`	이 블록은 한 번에 한 스레드만 실행합니다. 블록이 끝나면 잠금이 저절로 풀립니다
`_clock = time.monotonic`	함수를 **변수에 담아 둡니다**. 시험에서 손으로 움직이는 시계로 바꿔 끼우기 위해서입니다
`while q and now - q[0] > …: q.popleft()`	가장 오래된 기록부터 10분이 지난 것을 버립니다
`del _recent_sends[key]`	빈 목록은 열쇠째 지웁니다. 메모리가 계속 늘어나지 않게 하기 위해서입니다
`_recent_sends.setdefault(key, deque())`	열쇠가 없으면 빈 목록을 만들어 넣고, 있으면 그 목록을 돌려줍니다
`int(…) + 1`	남은 초를 올림합니다. 0초 남았다고 알려 주지 않기 위해서입니다

4. 흐름

검사: 지금 시각을 구하고 잠금을 겁니다. 열쇠(이메일, IP)마다 10분이 지난 기록을 버리고, 목록이 비면 지웁니다. 기록이 2개 이상이면 429로 응답합니다.

세기 (저장에 성공한 뒤): 지금 시각을 구하고 잠금을 건 다음, 열쇠마다 시각을 하나씩 추가합니다.

5. 왜 이렇게

선택	이유
검사와 세기를 둘로 나눔	"저장에 실패한 시도는 세지 않는다"는 사용자 결정에 따라, 세는 시점을 저장 뒤로 옮겼습니다
이메일 **또는** IP로 제한	이메일만 보면 주소만 바꿔서 계속 보낼 수 있고, IP만 보면 같은 사람이 와이파이를 바꿔 보낼 수 있습니다. 그래서 둘 다 봅니다
이메일은 소문자로 바꾸고 공백을 지워서 비교	`A@Example.com`과 `a@example.com`은 같은 사람이기 때문입니다

선택	이유
IP가 없으면 이메일로만 셈	시험 도구처럼 IP를 알 수 없는 요청을 위한 처리입니다. `ip:None`이라는 열쇠를 만들지 않아서, 모든 요청이 한 열쇠로 묶이는 일을 막습니다
`time.monotonic` 사용	서버 시계가 바뀌어도(시간 동기화 등) 10분 계산이 틀어지지 않습니다
잠금 사용	FastAPI의 동기 라우터는 여러 스레드에서 실행됩니다. 동시에 같은 목록을 고치면 기록을 잃을 수 있습니다 (시험: 스레드 100개)

알아 둘 점 (동시에 두 번 보내는 경우): 같은 사람이 **정확히 같은 순간에** 두 번 보내면, 둘 다 검사를 통과한 뒤 저장되어 한도보다 1번 더 저장될 수 있습니다. 검사부터 저장까지를 한 잠금으로 묶으면 막을 수 있지만, 그동안 다른 사람의 문의까지 기다려야 합니다. 문의 폼은 보내는 동안 버튼을 잠그기 때문에(화면) 실제로 일어날 일은 드뭅니다.

6. 시험 (`tests/test_inquiry_service.py` 새 파일, 12개, 시계를 손으로 움직이며 시험)

시험	확인한 것
상수	2번, 600초입니다 (사용자 결정)
3번째	429와 문구, `retry_after` 601초가 나옵니다
이메일이 같고 IP가 다름(대소문자와 공백도 다름), IP가 같고 이메일이 다름	둘 다 막힙니다
다른 사람	통과합니다
IP 없음	이메일로만 세고, `ip:None` 열쇠는 생기지 않습니다
검사만 5번 한 뒤(저장 실패) 실제 저장 2번	2번 모두 저장되고 3번째는 막힙니다. **실패한 시도는 세지 않습니다**
막힌 뒤 10분이 지남	다시 보낼 수 있습니다 (막힌 시도도 세지 않습니다)
`retry_after`	첫 기록을 기준으로 줄어듭니다
10분 범위가 밀림	첫 기록이 10분을 넘기면 1자리가 남습니다
오래된 기록	지워져서 기록 장소가 비게 됩니다
스레드 100개가 동시에 셈	기록을 하나도 잃지 않습니다

전체 시험 380개가 통과했고, `ruff` 검사도 통과했습니다.